Rapport de projet — Introduction aux Systèmes Distribués

Routage de messages SMS sur un réseau d'antennes

Benoît SION Kalvin-Giang PICHON

Résumé

Ce rapport présente la conception et l'implémentation d'un système distribué simulant le routage de messages SMS à travers un réseau d'antennes. Chaque antenne est une entité distribuée indépendante communiquant via RabbitMQ. Le routage repose sur un mécanisme de diffusion avec déduplication afin d'acheminer les paquets à travers la topologie du réseau.

1 Introduction

L'objectif de ce projet est de simuler le routage de messages SMS dans un réseau d'antennes géographiquement distribuées. Chaque antenne connaît sa position et son rayon de couverture, et peut communiquer avec ses voisines si leurs zones de couverture se chevauchent. Des téléphones mobiles peuvent se connecter à l'antenne la plus proche et envoyer des messages à d'autres téléphones, quel que soit leur emplacement dans le réseau.

1.1 Ce qui fonctionne

- Lancement de N antennes indépendantes à partir d'un fichier de configuration commun.
- Calcul automatique de la matrice d'adjacence selon la géométrie des antennes.
- Routage de messages entre antennes par diffusion avec déduplication (*broadcast flooding*).
- Lancement de téléphones mobiles se connectant à l'antenne la plus proche dans son rayon de couverture.
- Envoi et réception de SMS entre téléphones via le réseau d'antennes.
- Plusieurs téléphones peuvent être couverts par la même antenne.

1.2 Ce qui ne fonctionne pas / limitations connues

- La mobilité des téléphones n'est pas gérée : un téléphone est associé à une antenne à son lancement et ne peut pas en changer.
- Le réseau est supposé statique : aucune gestion de la panne d'une antenne ni de reconnexion dynamique.
- La scalabilité du flood est limitée : le nombre de messages transitant dans le réseau croît avec le nombre d'antennes et de paquets en circulation.
- RabbitMQ doit fonctionner sur `localhost` ; le système n'est pas encore configuré pour un déploiement multi-machines.

2 Architecture

2.1 Entités distribuées

Le système repose sur deux types d'entités distribuées indépendantes :

Antenne (`AntenneImplementation`) Entité centrale du réseau. Chaque antenne est un processus Java distinct. Elle connaît sa position (x, y) et son rayon de couverture r. Au démarrage, elle calcule ses voisines depuis la matrice d'adjacence et établit ses connexions RabbitMQ. Elle relaie les paquets qu'elle reçoit à ses voisines, ou les délivre localement si la destination est un téléphone qu'elle couvre.

Téléphone (`Phone`) Terminal client. Chaque téléphone est un processus Java distinct. Il se connecte à l'antenne la plus proche couvrant sa position géographique et peut envoyer et recevoir des SMS. Il n'a pas connaissance de la topologie du réseau : il se contente de publier ses messages sur l'exchange de l'antenne associée.

2.2 Hypothèses

- Toutes les entités (antennes et téléphones) partagent le même fichier de configuration décrivant la topologie. Ce fichier est lu au démarrage et ne change pas pendant l'exécution.
- Deux antennes sont voisines si et seulement si la distance euclidienne entre leurs centres est inférieure ou égale à la somme de leurs rayons : $d(A_i, A_j) \leq r_i + r_j$.
- Un téléphone est couvert par l'antenne la plus proche dont le rayon l'englobe.
- RabbitMQ est accessible sur `localhost:5672` pour toutes les entités.

2.3 Interactions et topologie RabbitMQ

La communication entre entités est entièrement médiée par RabbitMQ. Aucun appel direct d'une entité à l'autre n'existe dans le code applicatif.

Exchanges et queues des antennes. Chaque antenne i possède :

- Un *exchange fanout* nommé `ex-i`, sur lequel elle publie ses messages à diffuser. L'exchange fanout redistribue le message à *toutes* les queues qui lui sont liées, sans que l'émetteur n'ait à connaître leurs noms.
- Une *queue* personnelle nommée `queue-i`, liée aux exchanges de toutes ses antennes voisines. Ainsi, lorsqu'une voisine publie sur son exchange, l'antenne i reçoit le message dans sa queue.

L'antenne n'est *pas* liée à son propre exchange : un message qu'elle publie sur `ex-i` est reçu uniquement par ses voisines, et non par elle-même directement.

Queues des téléphones. Chaque téléphone de GlobalId g dispose d'une queue dédiée `queue-phone-g`, déclarée par l'antenne qui le couvre. C'est sur cette queue que l'antenne dépose les messages destinés à ce téléphone, et que le processus `Phone` écoute.

Lorsqu'un téléphone envoie un message, il publie directement sur l'exchange `ex-{antennaId}` de l'antenne la plus proche : le message entre ainsi dans le réseau d'antennes sans passer par une queue intermédiaire.

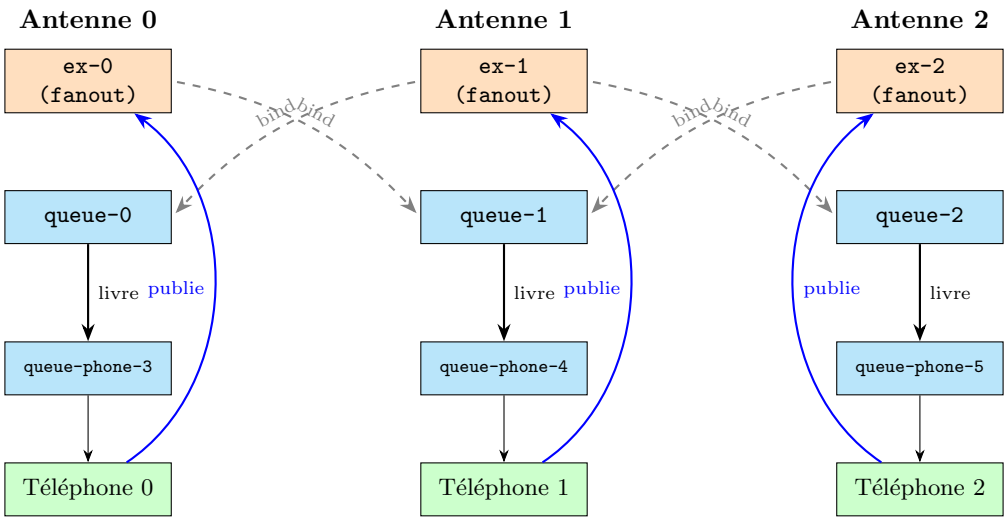

FIGURE 1 – Topologie RabbitMQ pour 3 antennes en ligne avec 3 téléphones. Chaque antenne possède un exchange fanout (orange) et une queue (bleu clair). Les flèches en tirets représentent les bindings : la queue d'une antenne est liée aux exchanges de ses voisines. Les téléphones (vert) publient sur l'exchange de leur antenne et écoutent leur queue de livraison dédiée.

Schéma de la topologie RabbitMQ.

2.4 Algorithme de routage

Le routage utilise une stratégie de **broadcast flooding avec déduplication**. Lorsqu'une antenne reçoit un paquet :

1. Elle vérifie si le paquet a déjà été traité, via une clé `(from, msgId)` stockée dans un ensemble concurrent (`ConcurrentHashMap.newKeySet()`). Si la clé est déjà présente, le paquet est ignoré (prévention des boucles).
2. Si la destination est l'identifiant d'un téléphone couvert localement, le paquet est déposé directement dans la queue `queue-phone-g` correspondante.
3. Sinon, le paquet est republié sur l'exchange `ex-i` de l'antenne (`broadcastMessage`), ce qui le diffuse à toutes ses voisines.

Prévention des écho structurels. Lorsqu'une antenne diffuse via `broadcastMessage`, elle enregistre d'abord la clé `(from, msgId)` dans `seenMessages` *avant* de publier sur l'exchange. Ainsi, lorsque les voisines relaient le même paquet en retour (écho inévitable avec le fanout flooding), l'antenne source le détecte et l'ignore immédiatement.

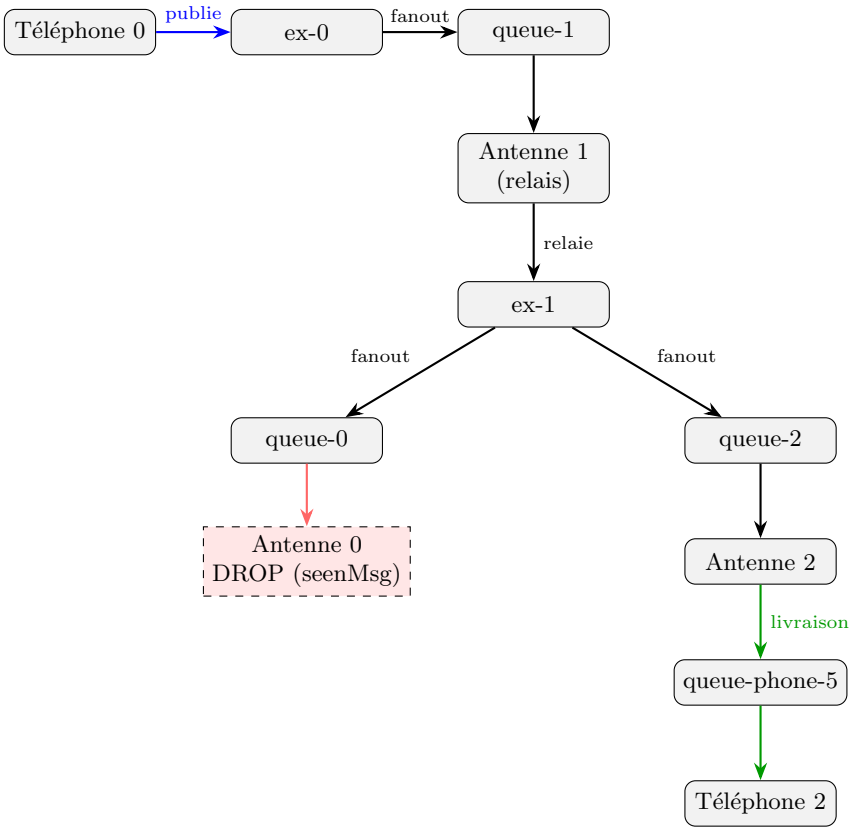

FIGURE 2 – Routage d'un SMS du Téléphone 0 (couvert par Antenne 0) vers le Téléphone 2 (couvert par Antenne 2), dans la configuration en ligne A0–A1–A2. L'écho reçu par Antenne 0 (flèche rouge) est silencieusement ignoré grâce au mécanisme de déduplication.

Diagramme de routage : de Téléphone 0 vers Téléphone 2.

2.5 Identifiants globaux

Pour différencier les antennes des téléphones dans les champs `to` et `from` des paquets, un espace d'identifiants global est défini :

— Antennes : identifiants $[0, N-1]$
— Téléphones : identifiants $[N, N+M-1]$, où N est le nombre d'antennes et M le nombre de téléphones.

Ce partitionnement permet à chaque antenne de déterminer sans ambiguïté si la destination d'un paquet est une antenne ($\texttt{to} < N$) ou un téléphone ($\texttt{to} \geq N$).

2.6 Justification des choix

— **Évolution de l'architecture.** La première implémentation utilisait $N \times (N-1)$ queues directes entre antennes (une queue par paire ordonnée émetteur/récepteur). Chaque antenne

connaissait explicitement les noms des queues de ses voisines. Cette approche fonctionnait mais couplait fortement les entités et rendait la configuration complexe. Nous avons ensuite refactorisé vers les exchanges fanout, ce qui réduit le nombre d'objets RabbitMQ à N exchanges et N queues, et découple totalement l'émetteur de ses destinataires.

— **Flood plutôt que spanning tree.** La simplicité de mise en œuvre a guidé ce choix. Un spanning tree aurait nécessité un protocole de consensus distribué pour construire l'arbre, ce qui constitue une problématique en soi. Le flood avec déduplication est correct par construction et ne nécessite pas de synchronisation globale.

— **Exchange fanout.** En publiant sur un exchange fanout, une antenne n'a pas besoin de connaître les noms des queues de ses voisines ; RabbitMQ gère la diffusion. La topologie est exprimée dans les bindings (déclarés au démarrage de chaque antenne), pas dans le code applicatif. Cela correspond naturellement au comportement de relais souhaité.

— **Processus séparés.** Chaque antenne et chaque téléphone est un processus JVM distinct, conformément au modèle distribué (pas de mémoire partagée entre entités).

3 Implémentation

3.1 Structure du code

Le projet contient cinq classes Java dans le répertoire `src/` :

Classe	Rôle
`AntenneInterface`	Interface définissant le contrat d'une antenne : `broadcastMessage(Packet)` et `initCommunication()`.
`AntenneImplementation`	Implémentation de l'antenne. Gère la connexion RabbitMQ, la déclaration des exchanges/queues et des bindings, l'écoute et le relayage des paquets. Contient également le `main` point d'entrée du processus antenne.
`Phone`	Processus téléphone. Présente la liste des téléphones disponibles, se connecte à l'antenne la plus proche, écoute sa queue de réception et envoie des paquets via l'exchange de l'antenne associée.
`Packet`	Modèle de données transporté dans le réseau. Contient l'émetteur, le destinataire, le message et un identifiant unique de message (`msgId`). Sérialisé en bytecode Java (`ObjectOutputStream`) pour le transport via RabbitMQ.
`Utils`	Utilitaires de configuration. Lit le fichier de topologie, calcule la matrice d'adjacence par géométrie euclidienne, et identifie l'antenne la plus proche d'un téléphone.

3.2 Format du fichier de configuration

Le fichier de configuration décrit la topologie du réseau. Il est lu par `Utils.readConfig()` :

```
N              # nombre d'antennes
x0 y0 r0       # position et rayon de l'antenne 0
x1 y1 r1       # position et rayon de l'antenne 1
...
M              # (optionnel) nombre de telephones
px0 py0        # position du telephone 0
px1 py1        # position du telephone 1
...
```

Exemple (`test/graphe-simple-ligne-utilisateurs`) :

```
3
0 0 2
3 0 2
6 0 2
3
0 0
3 0
6 0
```

Ce fichier définit 3 antennes en ligne avec un rayon de 2 unités chacune, et 3 téléphones positionnés sur chaque antenne. La section téléphones est optionnelle : si elle est absente, le réseau fonctionne en mode antenne-uniquement.

3.3 Déduplication des messages

La prévention des boucles est assurée par un `Set<String>` concurrent (`ConcurrentHashMap.newKeySet()`) stockant les clés `"from-msgId"` de chaque paquet traité. Cette structure est thread-safe car le thread d'écoute RabbitMQ et le thread principal peuvent y accéder simultanément.

La clé est insérée dans l'ensemble à deux moments :

1. **À la réception** (dans `deliverCallback`) : si la clé est déjà présente, le paquet est ignoré immédiatement.
2. **Avant l'émission** (dans `broadcastMessage`) : la clé est enregistrée *avant* de publier sur l'exchange, afin que l'écho éventuel retourné par les voisines soit reconnu et éliminé sans traitement superflu.

4 Déploiement

4.1 Prérequis

— Java 11 ou supérieur
— Docker (pour lancer RabbitMQ)
— Les bibliothèques `amqp-client.jar`, `slf4j-api.jar` et `slf4j-simple.jar` fournies dans le répertoire `lib/`

4.2 Compilation

Depuis la racine du projet :

```
javac -cp "lib/*" -d bin src/*.java
```

4.3 Lancement

1. **Démarrer RabbitMQ.**

   ```
   ./start-rmq
   ```

   Ce script lance un conteneur Docker RabbitMQ avec les ports par défaut (5672).

2. **Lancer les antennes.**

   Chaque antenne doit être lancée dans un terminal séparé (un processus par antenne) :

   ```
   java -cp bin:"lib/*" AntenneImplementation <id> <fichier_config>
   ```

   Exemple avec 3 antennes sur le graphe en ligne :

   ```
   # Terminal 1
   java -cp bin:"lib/*" AntenneImplementation 0 test/graphe-simple-ligne
   # Terminal 2
   java -cp bin:"lib/*" AntenneImplementation 1 test/graphe-simple-ligne
   # Terminal 3
   java -cp bin:"lib/*" AntenneImplementation 2 test/graphe-simple-ligne
   ```

3. **Lancer des téléphones (optionnel).**

   ```
   java -cp bin:"lib/*" Phone test/graphe-simple-ligne-utilisateurs
   ```

   Le processus affiche les téléphones disponibles et demande à l'utilisateur d'en choisir un. Une fois connecté, le format d'envoi est :

   ```
   phone> <message> <destLocalPhoneId>
   ```

4. **Envoi d'un message depuis une antenne.**

```
antenne> <id_destinataire> <message>
```

5. **Arrêter RabbitMQ.**

```
./stop-rmq
```

4.4 Modèle de distribution

Chaque entité (antenne ou téléphone) est un processus JVM indépendant, sans mémoire partagée avec les autres entités. Dans ce projet, toutes les entités tournent sur la même machine et se connectent au même broker RabbitMQ (`localhost`). L'architecture est cependant distribuable sur plusieurs machines en modifiant l'adresse du broker dans le code (`factory.setHost(...)`).

Le projet n'a pas été testé sur le serveur Mandelbrot. Il devrait néanmoins y fonctionner sans modification, à condition que RabbitMQ soit démarré sur la même machine ou que son adresse soit reconfigurée.

5 Gestion des erreurs

— Si RabbitMQ est inaccessible au démarrage, une exception est levée et le processus s'arrête avec un message d'erreur explicite.
— Si le fichier de configuration est mal formé (nombre d'antennes incorrect, champs manquants), une `IOException` ou `NullPointerException` est levée avec un message descriptif.
— Si un téléphone se trouve hors couverture de toutes les antennes, le processus `Phone` s'arrête avec un message d'erreur.
— Les paquets en double sont silencieusement ignorés (log `[~] Duplicate dropped`).

6 Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux des systèmes distribués : entités autonomes sans mémoire partagée, communication asynchrone via un middleware de messagerie, et gestion des problèmes propres à la distribution (boucles, doublons, topologie dynamique).

Difficultés rencontrées. La principale difficulté a été la gestion des écho structurels inhérents au broadcast flooding avec des exchanges fanout. En diffusant via `ex-0`, l'antenne 0 provoque le relais d'Antenne 1 via `ex-1`, dont la queue de l'antenne 0 est cliente. Antenne 0 reçoit donc son propre message en retour. Ce comportement est inévitable avec cette topologie : la solution adoptée — enregistrer la clé `(from, msgId)` dans `seenMessages` avant de publier — s'est avérée simple et efficace.

Une autre difficulté est venue de l'architecture initiale à files directes ($N \times (N-1)$ queues). Bien que fonctionnelle, elle couplait fortement les entités et rendait difficile l'extension du système. La refactorisation vers les exchanges fanout a sensiblement simplifié le code et l'a rendu plus fidèle au modèle distribué.

Apports et réflexions. L'utilisation de RabbitMQ s'est révélée très adaptée à ce problème : la notion d'exchange fanout correspond naturellement au comportement de diffusion à tous les voisins, et le découplage entre producteur et consommateur simplifie considérablement la gestion des connexions. Nous avons appris à identifier les primitives du middleware qui correspondent aux besoins du protocole, plutôt que d'en réimplémenter la logique manuellement.

Le choix du broadcast flooding est fonctionnel mais présente des limites de performance évidentes. Une amélioration naturelle serait d'implémenter un protocole de routage à vecteur de distances, ce qui nécessiterait un échange périodique de tables de routage entre voisines — une problématique distribuée nettement plus complexe mais plus scalable.